

Лабораторная работа №2

«Проектирование архитектуры и технологической основы выпускной квалификационной работы»

1. Требования к приложению

В рамках выпускной квалификационной работы разрабатывается веб-приложение для кофейни — система предварительных заказов с личным кабинетом, бонусной программой и административной панелью.

Приложение должно поддерживать три роли пользователей:

- **Гость** — просмотр меню, новостей и мероприятий без регистрации;
- **Зарегистрированный клиент** — просмотр бонусного счёта и QR-кода, оформление предзаказа, оплата бонусами;
- **Сотрудник** — редактирование меню, новостей и фотогалереи;
- **Администратор** — доступ к метрикам, управление всем контентом.

Система должна работать как SPA (Single Page Application) с отдельным REST API-бэкендом.

2. Используемые технологии

В ходе анализа доступных инструментов был выбран современный полный стек на основе языка JavaScript/TypeScript — единого языка как для клиентской, так и для серверной части. Это решение позволяет снизить когнитивную нагрузку при разработке и ускорить процесс создания приложения.

Фронтенд:

Технология	Назначение
React 19	UI-фреймворк, компонентный интерфейс
Vite + SWC	Инструмент сборки, быстрая горячая перезагрузка
Tailwind CSS v4	Утилитарная стилизация компонентов
RizzUI	Библиотека готовых минималистичных UI-компонентов

React Router v7	Клиентская маршрутизация (SPA)
Zustand	Управление глобальным состоянием
React Hook Form + Zod	Управление формами и валидация данных
Axios+ReactQuery	HTTP-клиент для запросов к API

Бэкенд:

Технология	Назначение
Node.js	Серверная платформа выполнения JavaScript
TypeScript	Статическая типизация
Express	HTTP-фреймворк, маршрутизация запросов
Prisma ORM	Типобезопасная работа с базой данных
PostgreSQL	Реляционная система управления базами данных
JWT + bcrypt	Аутентификация и хэширование паролей
dotenv	Управление переменными окружения

Инструменты разработки:

- **Git + GitHub** — система контроля версий и удалённый репозиторий;
- **VS Code** — среда разработки;
- **Thunder Client** — тестирование API-запросов;
- **pgAdmin 4** — администрирование базы данных.

3. Архитектура приложения

Приложение построено по клиент-серверной архитектуре с чётким разделением на два независимых проекта. Фронтенд взаимодействует с бэкендом исключительно через REST API, что обеспечивает независимость слоёв и возможность их раздельного развёртывания.

Бэкенд организован по принципу **Layered Architecture** (слоистая архитектура):

- **Routes** — принимают HTTP-запросы и передают их в контроллеры;

- **Controllers** — содержат бизнес-логику обработки запросов;
- **Middleware** — промежуточные обработчики (проверка JWT-токена, валидация);
- **Prisma** — слой доступа к данным, взаимодействие с PostgreSQL.

Фронтенд организован по принципу **Feature-based** структуры:

- **pages/** — страницы приложения (главная, меню, личный кабинет, панель администратора);
- **components/** — переиспользуемые UI-компоненты;
- **store/** — глобальное состояние через Zustand;
- **api/** — модули запросов к бэкенду через Axios.

Схема взаимодействия компонентов:

text

Браузер (React SPA)

↓ HTTP-запросы (JSON)

Express REST API (Node.js)

↓ Prisma ORM

PostgreSQL Database

4. Описание стартового проекта

Проект инициализирован в виде монорепозитория — единой папки `diplom/`, содержащей два независимых подпроекта.

Оба проекта успешно запускаются: фронтенд — на `http://localhost:5173`, бэкенд — на `http://localhost:3001`. Базовая точка входа бэкенда принимает запросы и возвращает ответ. На фронтенде отображается тестовый компонент библиотеки RizzUI, подтверждающий корректность подключения всех зависимостей.

Инструкция по запуску:

bash

Бэкенд

```
cd backend
```

```
npm install
```

```
npm run dev
```

```
# Фронтенд (в отдельном терминале)
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

5. Ссылка на удалённый репозиторий

<https://github.com/dimasoko/diplom>